Heuristic Search Algorithm

Analog Circuit Optimization: Algorithmic Search & Fine-Tuning

MSc Thesis

July 2026

The Analog Optimization Problem

In analog integrated circuit design, we define the circuit parameters as an input vector $\vec{x}$ and the resulting specifications as an output vector $\vec{y}$. The relationship is governed by a complex function space $\vec{y} = f(\vec{x})$.

$$\vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \vec{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = f(\vec{x})$$

Consider a scenario where the entire $\vec{y}$ vector achieves its target specifications (TG), except for a single component:

- y_1 achieves 1^{st} TG
- y_2 achieves 2^{nd} TG
- ...
- y_j **does not** achieve j^{th} TG
- ...
- y_m achieves m^{th} TG

We are constrained to fine-tune only one component in $\vec{x}$. We possess prior knowledge of the gradient effects of every component in $\vec{x}$ on every component in $\vec{y}$ (e.g., increasing parameter x_{10} decreases specification y_5). The objective is to systematically fine-tune a component x_i such that all components in $\vec{y}$ achieve their specified targets simultaneously.

Algorithmic Fine-Tuning

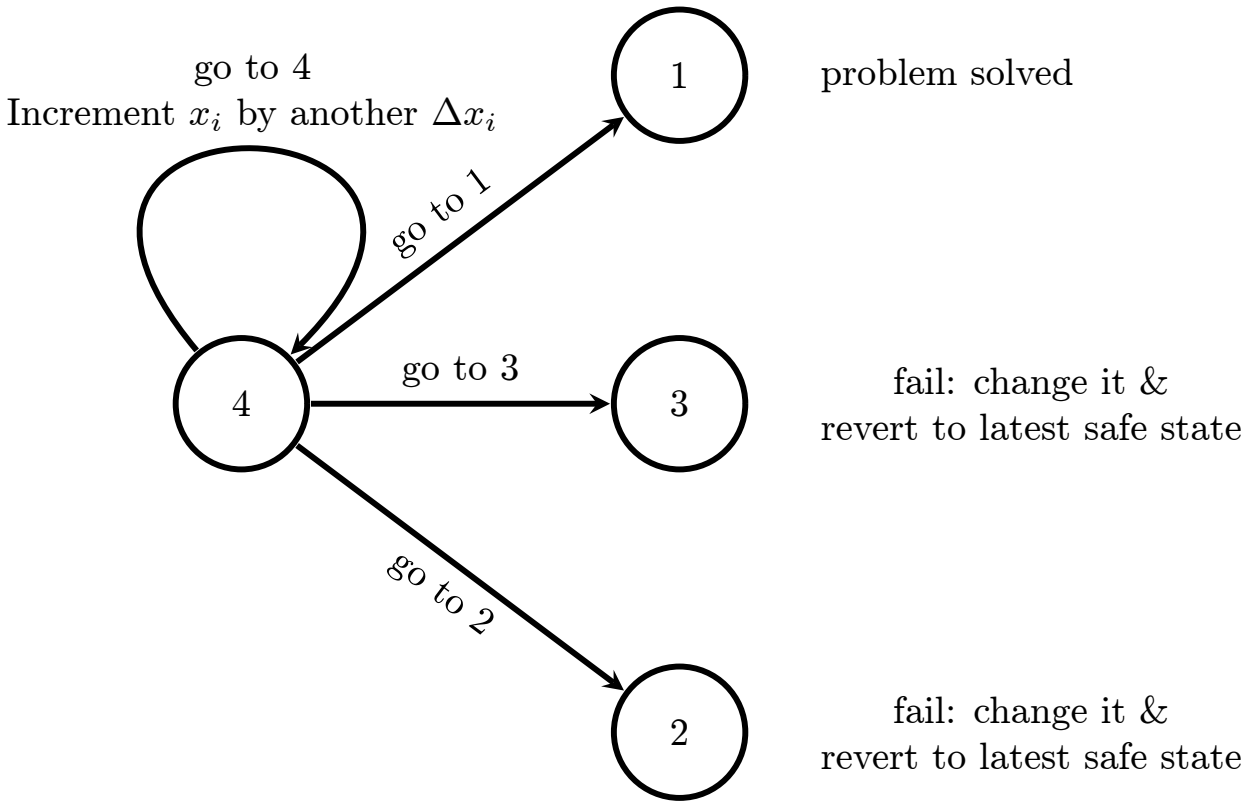
1. The Normal Algorithm

Assume we select a component x_i to vary in order to improve the failing specification y_j. We apply a small perturbation:

$$x_i(t+1) = x_i(t) + \Delta x_i \quad (\text{where } \Delta x_i \text{ is very small})$$

This yields four possible outcomes (Nodes):

1. y_j achieves its spec, and all other y_k (where $k \neq j$) still achieve their targets. *(Problem Solved)*
2. y_j achieves its spec, but not all y_k achieve their targets anymore. *(Must change x_i component to another one)*
3. y_j improves, but not all y_k achieve their targets anymore. *(Must change x_i component to another one)*
4. y_j improves, and all y_k still achieve their target specs. *(Increment x_i by another Δx_i)*

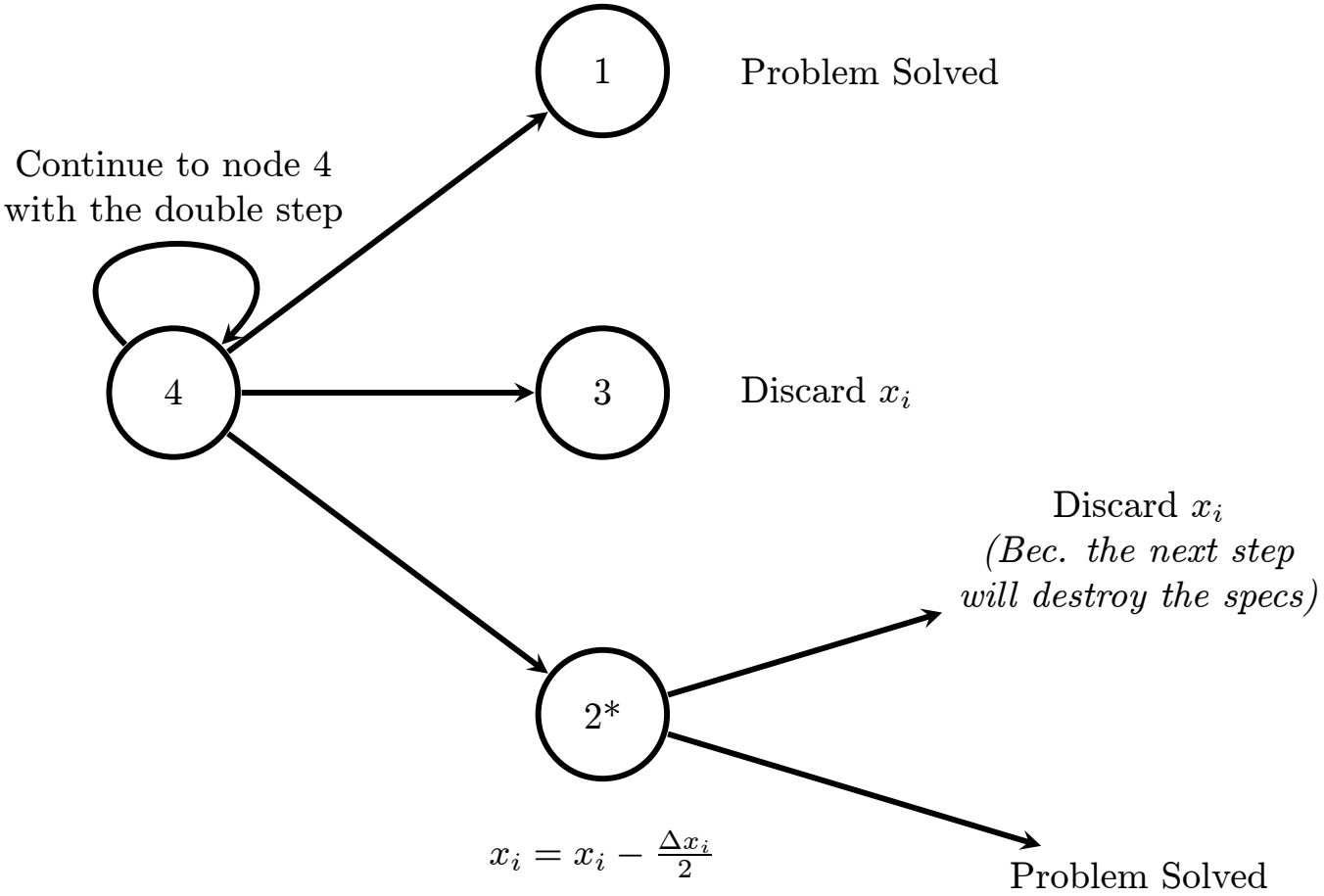


2. The Improved Algorithm

A faster algorithm dictates that if you have passed by Node 4 twice before and this is your 3rd visit to it, you double your step size: $\Delta x_i = 2\Delta x_i$[cite: 3].

However, increasing the step size introduces overshoot risks, requiring a modification to Node 2[cite: 3]:

- **Modified State (2*):** y_j achieves spec, but not all other y_k achieve their spec[cite: 3]. We cannot simply discard x_i because the doubled step size might have improved y_j too aggressively, pushing other specs out of bounds due to being too large[cite: 3]. Instead of discarding x_i, we halve the step size and decrement: $x_i = x_i - \frac{\Delta x_i}{2}$[cite: 3].



3. Programmatic Adaptive Step-Size Search

The fully automated implementation of the heuristic search relies on dynamic step-size modulation (acceleration and backtracking) based on simulation feedback.

The algorithm initializes a base step percentage of $\Delta x = 5\%$ (0.05) and evaluates a perturbed parameter x_{test}:

$$x_{test} = x_{safe} \cdot (1 \pm \Delta x)$$

Upon simulation, the algorithm evaluates the specifications and triggers one of the following topological outcomes:

1. **Target Achieved (Success):** The target specification hits its required value while all constraints remain valid. The algorithm terminates successfully.
2. **Gradient Stable (Acceleration):** The target specification improves by $\geq 1\%$ and all other constraints pass. The new safe state is saved. If this successful improvement occurs ≥ 2 consecutive times, the algorithm assumes a stable gradient and doubles the step size to accelerate convergence:

$$\Delta x_{new} = 2 \cdot \Delta x_{old}$$

3. **Overshoot / Crash (Backtracking):** The simulation pushes the geometry into an invalid state (crash) or violates the established constraints. The algorithm rejects the step, halves the step size, and resets the acceleration counter:

$$\Delta x_{new} = \frac{\Delta x_{old}}{2}$$

   If the step size falls below the 0.5% *threshold (*$\Delta x < 0.005$*) during backtracking, the variable is considered exhausted and is discarded.*

4. **Plateau (Discard):** The target specification fails to improve by at least 1% and the target is not hit. To conserve the simulation

budget, the variable is immediately discarded.